

Question Paper

Exam Date & Time: 16-Nov-2024 (01:15 PM - 04:30 PM)



CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY

University Examination Nov 2024
Seventh Semester of B.Tech. (CS)

Design of Language Processor [CS450]

Marks: 70

Duration: 195 mins.

Section - 1

Answer all the questions.

Section Duration: 40 mins

1

_____ is the first software to run when the computer is switched on

(1)

Operating system

Bootstrap loader

Linking loader

Absolute loader

Course Outcome: "CO1" / Cognitive Level: "Remember"

2

The number of tokens in the following C statement is

(1)

printf("HELLO WORLD");

3

5

9

8

Course Outcome: "CO1" / Cognitive Level: "Understand"

3

Which one of the following statements is FALSE?

(1)

Context-free grammar can be used to specify both lexical and syntax rules.

Type checking is done before parsing.

High-level language programs can be translated to different Intermediate Representations.

Arguments to a function can be passed using the program stack.

Course Outcome: "CO2" / Cognitive Level: "Remember"

4

Consider the following statements related to compiler construction :

(1)

I. Lexical Analysis is specified by context-free grammars and implemented by push-down automata.

II. Syntax Analysis is specified by regular expressions and implemented by finite-state machine.

Which of the above statement(s) is/are correct ?

Only I

Only II

Both I and II

Neither I nor II

Course Outcome: "CO3" / Cognitive Level: "Application"

5

Which of the following statement(s) regarding a linker software is/are true ?

(1)

- 1) A function of a linker is to combine several object modules into a single load module.
2) A function of a linker is to replace absolute references in an object module by symbolic references to locations in other modules.

[Only I](#) [Only II](#) [Both I and II](#) [Neither I nor II](#)

Course Outcome: "CO1" / Cognitive Level: "Remember"

6

Debugger is a program that

(1)

[allows to examine and modify the contents of registers](#) [does not allow execution of a segment of program](#) [allows to set breakpoints, execute a segment of program and display contents of register](#) [All of the above](#)

Course Outcome: "CO1" / Cognitive Level: "Remember"

7

Match all items in Group 1 with correct options from those given in Group 2.

(1)

Group 1	Group 2
P. Regular expression	1. Syntax analysis
Q. Pushdown automata	2. Code generation
R. Dataflow analysis	3. Lexical analysis
S. Register allocation	4. Code optimization

[P-4, Q-1, R-2, S-3](#) [P-3, Q-1, R-4, S-2](#) [P-3, Q-4, R-1, S-2](#) [P-2, Q-1, R-4, S-3](#)

Course Outcome: "CO2" / Cognitive Level: "Remember"

8

An LALR(1) parser for a grammar G can have shift-reduce (S-R) conflicts if and only if

(1)

[the SLR\(1\) parser for G has S-R conflicts](#) [the LR\(1\) parser for G has S-R conflicts](#) [the LR\(0\) parser for G has S-R conflicts](#) [the LALR\(1\) parser for G has reduce-reduce conflicts](#)

Course Outcome: "CO4" / Cognitive Level: "Understand"

9

Consider the grammar with non-terminals $N = \{S, C, S_1\}$, terminals $T = \{a, b, i, t, e\}$, with S as the start symbol, and the following set of rules:

$S \rightarrow iCtSS_1|a$

$S_1 \rightarrow eS|\epsilon$

$C \rightarrow b$

The grammar is NOT LL(1) because:

[it is left recursive](#) [it is right recursive](#) [it is ambiguous](#) [It is not context-free.](#)

Course Outcome: "CO3" / Cognitive Level: "Analysis"

10

Assume that the SLR parser for a grammar G has n_1 states and the LALR parser for G has n_2 states. The relationship between n_1 and n_2 is: (1)

[n1 is necessarily less than n2](#)

[n1 is necessarily equal to n2](#)

[n1 is necessarily greater than n2](#)

[none of these](#)

Course Outcome: "CO3" / Cognitive Level: "Application"

11

Which of the following derivations does a top-down parser use while parsing an input string? The input is assumed to be scanned in left to right order. (1)

[Leftmost derivation traced out in reverse](#)

[Rightmost derivation](#)

[Leftmost derivation](#)

[Rightmost derivation traced out in reverse](#)

Course Outcome: "CO1" / Cognitive Level: "Understand"

12

Some code optimizations are carried out on the intermediate code because (1)

[they enhance the portability of the compiler to other target processors](#)

[program analysis is more accurate on intermediate code than on machine code](#)

[the information from dataflow analysis cannot otherwise be used for optimization](#)

[the information from the front end cannot otherwise be used for optimization](#)

Course Outcome: "CO5" / Cognitive Level: "Remember"

13

Among simple LR (SLR), canonical LR, and look-ahead LR (LALR), which of the following pairs identify the method that is very easy to implement and the method that is the most powerful, in that order? (1)

[SLR, LALR](#)

[Canonical LR, LALR](#)

[SLR, canonical LR](#)

[LALR, canonical LR](#)

Course Outcome: "CO4" / Cognitive Level: "Understand"

14

(1)

- (A) $S \rightarrow AB$
 $A \rightarrow Aa \mid b$
 $B \rightarrow c$
- (B) $S \rightarrow Ab \mid Bb \mid c$
 $A \rightarrow Bd \mid \epsilon$
 $B \rightarrow e$
- (C) $S \rightarrow Aa \mid B$
 $A \rightarrow Bb \mid Sc \mid \epsilon$
 $B \rightarrow d$
- (D) $S \rightarrow Aa \mid Bb \mid c$
 $A \rightarrow Bd \mid \epsilon$
 $B \rightarrow Ae \mid \epsilon$

Which one of the following grammars is free from left recursion?

[A](#) [B](#) [C](#) [D](#)

Course Outcome: "CO3" / Cognitive Level: "Analysis"

15

Which is True about SR and RR-conflict:

(1)

[If there is no SR-conflict in CLR\(1\) then definitely there will be no SR-conflict in LALR\(1\).](#)

[RR-conflict might occur if lookahead for final items\(reduce-moves\) is same.](#)

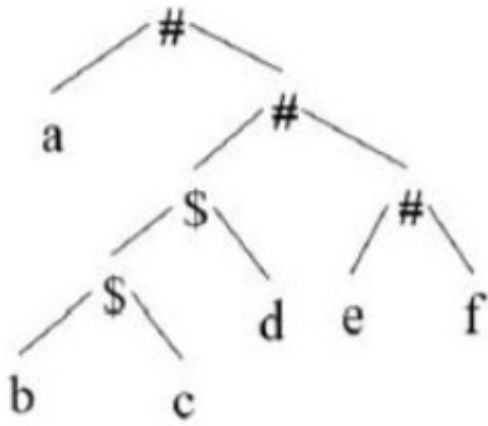
[Known that CLR\(1\) has no RR-conflict, still RR-conflict might occur in LALR\(1\).](#)

[All of the above.](#)

Course Outcome: "CO4" / Cognitive Level: "Understand"

16

Consider the following parse tree for the expression $a\#b\ c\ d\#e\#f$, involving two binary operators $\$$ and $\#$. (1)



Which one of the following is correct for the given parse tree?

- | | | | |
|--|--|---|--|
| \$ has higher precedence and is left associative; # is right associative | # has higher precedence and is left associative; \$ is right associative | \$ has higher precedence and is left associative; # is left associative | # has higher precedence and is right associative; \$ is left associative |
|--|--|---|--|

Course Outcome: "CO3" / Cognitive Level: "Application"

17

Yacc stands for

- | | | | |
|--|--|--|---|
| yet accept compiler constructs | yet accept compiler compiler | yet another compiler construct | yet another compiler compiler |
|--|--|--|---|

(1)

Course Outcome: "CO5" / Cognitive Level: "Remember"

18

Consider a macro defined with parameters. If the macro is invoked with more arguments than the parameters defined, what will most likely happen during compilation? (1)

- | | | | |
|--|--|---|---|
| The extra arguments will be ignored. | The macro will produce an error. | The compiler will attempt to merge extra arguments. | The macro will execute with default values for the extra arguments. |
|--|--|---|---|

Course Outcome: "CO1" / Cognitive Level: "Understand"

19

What is the main objective of a simple code generator in a compiler?

(1)

- | | | | |
|--|--|---|--|
| To translate high-level code into a target programming language. | To perform advanced optimization techniques. | To generate intermediate code from source code. | To translate intermediate representation to target machine code efficiently. |
|--|--|---|--|

Course Outcome: "CO5" / Cognitive Level: "Remember"

20

In flow graphs, which of the following statements describes a *back edge*?

(1)

- | | | | |
|---------------------------------------|---|------------------------------|---|
| An edge that connects | An edge that points back to | An edge that | An edge that only appears |
|---------------------------------------|---|------------------------------|---|

[two basic blocks in sequence.](#)

[a previous block, often indicating a loop.](#)

[does not impact control flow.](#)

[in the final code generation phase.](#)

Course Outcome: "CO5" / Cognitive Level: "Remember"

Section - 2

Answer 5 out of 8 questions.

1		
---	--	--

Construct SLR Parsing Table for the following grammar.

(4)

$S \rightarrow OS0 \mid 1S1 \mid 10$

Course Outcome: "CO3" / Cognitive Level: "Application"

2		
---	--	--

Analyze the different issues encountered in the design of a code generator and explain how each issue affects the code generation process. (4)

Course Outcome: "CO2" / Cognitive Level: "Analysis"

3		
---	--	--

Construct LL(1) parsing table for the following Grammar:

(4)

$S \rightarrow (L) \mid a$
 $L \rightarrow L, S \mid S$

Course Outcome: "CO3" / Cognitive Level: "Application"

4		
---	--	--

Compare and contrast top-down parsing (LL) and bottom-up parsing (LR) techniques. Discuss their strengths and weaknesses, particularly in terms of handling left recursion, ambiguity, and look ahead. Provide examples to illustrate your points. (4)

Course Outcome: "CO2" / Cognitive Level: "Understand"

5		
---	--	--

Discuss about S-attributed and L-attributed SDT with example.

(4)

Course Outcome: "CO4" / Cognitive Level: "Remember"

6		
---	--	--

Apply your understanding of synthesized and inherited attributes by defining each and providing an example that demonstrates their use in a syntax tree. (4)

Course Outcome: "CO2" / Cognitive Level: "Application"

7		
---	--	--

Describe the structure and purpose of three-address code, and explain how control flow is represented and managed within this format. Illustrate with examples. (4)

Course Outcome: "CO4" / Cognitive Level: "Analysis"

8		
---	--	--

Describe the role of intermediate code generation in a compiler. Discuss different variants of syntax trees and their significance in code optimization. (4)

Course Outcome: "CO5" / Cognitive Level: "Understand"

Section - 3

Answer 6 out of 9 questions.

1		
---	--	--

Analyze the input, output, and operations performed by each phase of a compiler, providing an example to illustrate each phase. (5)

Course Outcome: "CO1" / Cognitive Level: "Application"

2		
---	--	--

Examine the concept of a symbol table and analyze the various ways in which a compiler uses it. (5)

Course Outcome: "CO1" / Cognitive Level: "Application"

3		
---	--	--

Assess the accuracy of the First and Follow sets for the given grammar and provide a detailed explanation of each step in your solution. (5)

$S \rightarrow AaB \mid bC$

$A \rightarrow cD \mid \epsilon$

$B \rightarrow e \mid \epsilon$

$C \rightarrow f \mid \epsilon$

$D \rightarrow g \mid \epsilon$

Course Outcome: "CO3" / Cognitive Level: "Create"

4		
---	--	--

Check whether the following grammar is CLR or not. (5)

$S \rightarrow Aa \mid bAc \mid Bc \mid bBa$

$A \rightarrow d$

$B \rightarrow d$

Course Outcome: "CO3" / Cognitive Level: "Evaluate"

5		
---	--	--

Compare and analyze the Semantic Analysis and Syntax Analysis phases of a compiler, providing suitable examples for each. Also, discuss how each phase handles error reporting. (5)

Course Outcome: "CO4" / Cognitive Level: "Understand"

6		
---	--	--

Discuss the challenges of generating machine code for modern computer architectures, especially in the context of pipelining, parallelism, and multi-core processors. How do modern compilers optimize code to take advantage of these architectural features? (5)

Course Outcome: "CO5" / Cognitive Level: "Application"

7		
---	--	--

Symbol Table is an important data structure created and maintained by the compiler. (5)
What information is stored in symbol table? Also explain which data structures are used to implement symbol table with its pros and cons.

Course Outcome: "CO4" / Cognitive Level: "Understand"

8		
---	--	--

Show the following grammar is LR(1) but not LALR(1). (5)

$S \rightarrow Aa \mid bAc \mid Bc \mid bBa$

$A \rightarrow d$

$B \rightarrow d$

9		
---	--	--

What is DAG? What are its advantages in context of optimization? How does it help in eliminating (5) common sub expression?

-----End-----

Event Name		Subject	
University Examination Nov -Dec-2024		CS450:Design of Language Processor	
#	Course Outcome	Description	Marks
1	CO1	Understand the fundamentals of the compiler identify the relationships among different phases of the compiler and use the knowledge of the Lex tool	16
2	CO2	Describe the Role of Parser and the various error recovery strategies	14
3	CO3	Develop the parsers and experiment with the knowledge of different parser designs.	28
4	CO4	Develop a semantic analysis scheme to generate intermediate code	26
5	CO5	Summarize various optimization techniques used for dataflow analysis and generate machine code from the source code	13
Total			97

* NA - CO Not Selected

Event Name		Subject	
University Examination Nov -Dec-2024		CS450:Design of Language Processor	
#	Level	No of Questions	Marks (%)
1	Remember	10	13(13.4%)
2	Understand	11	29(29.9%)
3	Application	10	35(36.08%)
4	Analysis	4	10(10.31%)
5	Create	1	5(5.15%)
6	Evaluate	1	5(5.15%)
Total		37	97 (100%)